PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification

For Dehallucination Add-On Module for Enhancing Reliability of AI Applications

Version 1.0 approved

Team details

Team Name	Project ID selected	Team Member 1 (Name, SRN)	Team Member 2 (Name, SRN)	Team Member 3 (Name, SRN)	Team Member 4 (Name, SRN)
Hallucinators	39	Chinmay N S PES2UG24AM047	Badri Harinivas PES2UG24AM037	Ashrit Nair PES2UG24AM032	Aditya S PES2UG24AM015

Prepared by: Hallucinators
Organization: PES University, Department of Computer Science and Engineering
Date created: 5/09/2026

Table of Contents

Table of Contents....2
Revision History....3
1. Introduction....4
1.1 Purpose....4
1.2 Intended Audience and Reading Suggestions....4
1.3 Product Scope....4
1.4 References....5
2. Overall Description....5
2.1 Product Perspective....5
2.2 Product Functions....5
2.3 User Classes and Characteristics....5
2.4 Operating Environment....6
2.5 Design and Implementation Constraints....6
2.6 Assumptions and Dependencies....6
3. External Interface Requirements....7
3.1 User Interfaces....7
3.2 Software Interfaces....7
3.3 Communications Interfaces....7
4. Analysis Models....7
5. System Features....8
5.1 Claim Extraction....8
5.2 Fact Verification Engine....8
5.3 Confidence Scoring and Flagging....8
5.4 Correction and Citation Suggestion....9
5.5 Integration API / SDK....9
5.6 Reviewer Dashboard and Analytics....10
6. Other Nonfunctional Requirements....10
6.1 Performance Requirements....10
6.2 Safety Requirements....10
6.3 Security Requirements....10
6.4 Software Quality Attributes....11
6.5 Business Rules....11
7. Other Requirements....11
Appendix A: Glossary....11
Appendix B: Field Layouts....12
Appendix C: Requirement Traceability Matrix....12

Revision History

Name	Date	Reason For Changes	Version
Hallucinators (Team)	25/08/2026	Initial draft of SRS	0.1
Hallucinators (Team)	05/09/2026	Reviewed and approved	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Dehallucination Add-On Module (project code name: DAM), Version 1.0, developed as part of the UE24CS341A Software Engineering mini-project (Jackfruit) at PES University. This SRS covers the complete scope of the module: a pluggable service that intercepts text generated by a Large Language Model (LLM) or other AI content generator, extracts the factual claims within it, verifies each claim against trusted external sources and/or a curated knowledge base, and returns the original text annotated with confidence scores, flags on unsupported claims, and suggested corrections or citations. This document describes only the Dehallucination Add-On Module itself; it does not specify the internal design of any third-party LLM or chatbot that integrates with it.

1.2 Intended Audience and Reading Suggestions

This SRS is intended for the following readers:

- Development team members — to understand the functional and non-functional requirements they must implement.
- Course instructors and evaluators — to assess requirement coverage, testability, and traceability.
- Integrating developers (the users of the module's API/SDK) — to understand what inputs the module expects and what outputs it guarantees.
- Testers — to derive test cases directly from Section 5 (System Features) and Appendix C (Traceability Matrix).

Readers are advised to first review Section 2 (Overall Description) for context, then Section 5 (System Features) for the itemized functional requirements referenced throughout later development phases (Project Plan, Test Plan, Final Report).

1.3 Product Scope

Modern generative AI applications (chatbots, summarizers, Q&A assistants, content generators) occasionally produce statements that are fluent and confident but factually incorrect or unverifiable — a phenomenon commonly called "hallucination." This erodes user trust and can cause real-world harm when such output is used for decision-making. The Dehallucination Add-On Module aims to reduce this risk by acting as a verification and reliability layer that any AI application can call before showing generated text to an end user.

Objectives of the module:

- Automatically identify verifiable factual claims within a block of AI-generated text.
- Cross-check each claim against trusted knowledge sources (structured knowledge graphs, curated document corpora, and/or live web sources).
- Assign each claim a confidence/hallucination score and classify it as Supported, Unsupported, or Contradicted.
- Return an annotated response (with citations where available) that the calling application can render or act upon.
- Expose these capabilities through a simple, well-documented REST API/SDK so it can be integrated into existing chatbots and content-generation pipelines with minimal changes.

Out of scope for Version 1.0: real-time voice hallucination detection, multi-modal (image/video) claim verification, and automatic silent rewriting of the AI application's original output (the module flags and suggests; it does not unilaterally alter the source application's response).

1.4 References

- PES University, Department of CSE — "Jackfruit Mini-Project Software Engineering Guidelines," UE24CS341A, 2026.
- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications (template basis).
- Ji, Z. et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, 2023.
- Project Team GitHub Repository — <repository URL, to be added once the team repository is created>.
- Wikipedia REST API documentation — https://www.mediawiki.org/wiki/API:REST_API
- Wikidata Query Service documentation — https://query.wikidata.org/

2. Overall Description

2.1 Product Perspective

The Dehallucination Add-On Module is a new, self-contained, pluggable service. It is not a replacement for an LLM; rather, it sits alongside or downstream of one, receiving generated text as input and returning an enriched/annotated verification report as output. It can be deployed in three typical configurations:

- As a middleware proxy between the AI application backend and the end-user-facing interface (inline verification before display).
- As an asynchronous verification microservice called after a response is generated, with results surfaced in a review dashboard (post-hoc verification, e.g., for content moderation).
- As a directly embedded SDK/library call within an existing chatbot or content-generation pipeline.

A high-level context diagram: [AI Application / Chatbot / Content Generator] → generates text → [Dehallucination Add-On Module: Claim Extraction → Fact Verification Engine → Confidence Scoring → Annotation] → [Trusted Knowledge Sources: Knowledge Graph, Curated Corpus, Web Search] → annotated response → [End User / Reviewer Dashboard]. A formal context diagram will be produced as part of the High-Level Architecture document (Jackfruit Phase — Design).

2.2 Product Functions

At a high level, the module lets a calling application or a human reviewer:

- Submit a block of AI-generated text (with optional source/context metadata) for verification.
- Receive back the text segmented into individual claims, each tagged Supported / Unsupported / Contradicted / Not-Checkable, with a numeric confidence score.
- Receive citations or supporting evidence snippets for Supported claims.
- Receive suggested corrections for Contradicted claims where available.
- Configure which knowledge sources are trusted and prioritized (administrator function).
- View aggregate analytics on hallucination rates over time via a reviewer dashboard.
- Manage and query an internal cache/history of previously verified claims to avoid redundant lookups.

2.3 User Classes and Characteristics

User Class	Characteristics
Integrating Developer	Technical user who calls the module's REST API/SDK from their own chatbot or AI application. Needs clear API documentation, predictable response schemas, and low integration effort. Highest-frequency user class.

User Class	Characteristics
Content Reviewer / Moderator	Non-technical to semi-technical user who uses the web dashboard to review flagged claims, confirm or override verdicts, and approve/reject AI-generated content before publication.
System Administrator	Configures trusted knowledge sources, manages API keys/access, sets confidence thresholds, and monitors system health. Lowest-frequency, highest-privilege user class.
End User (indirect)	The ultimate consumer of the AI application's output; does not interact with the module directly but benefits from more reliable, citation-backed responses when the calling application chooses to surface annotations.

2.4 Operating Environment

- Backend service: Python 3.11+ (FastAPI) or Node.js, containerized with Docker, deployable on any Linux-based cloud VM or container orchestration platform (e.g., Kubernetes).
- Dashboard: Web-based single-page application (React), accessible via any modern evergreen browser (Chrome, Edge, Firefox) on desktop or tablet.
- Datastore: A relational database (PostgreSQL) for claim history, user accounts, and configuration; a vector database (e.g., FAISS/Chroma) for semantic search over the curated trusted-document corpus.
- External dependencies: Network access to public knowledge APIs (Wikipedia, Wikidata) and, optionally, a licensed web-search API, subject to their rate limits and terms of service.

2.5 Design and Implementation Constraints

- Must be built and delivered using the Agile methodology, with GitHub (Issues/Projects or a similar tool) actively tracking user stories and backlog, per course requirements.
- Must expose a CI/CD pipeline with automated code review, unit tests, and a security/static-analysis check on every pull request.
- The verification pipeline must not introduce unbounded latency; a per-request timeout and fallback behavior (e.g., "Not-Checkable" verdict) is required when a knowledge source does not respond in time.
- Must operate within the free/education tiers of any third-party APIs used during the course project (e.g., Wikipedia API rate limits), documented explicitly.
- Team size constraint: 3–4 students; each member must own one full functional feature end-to-end, verifiable via individual git commit history.
- Only English-language text is guaranteed to be supported in Version 1.0.

2.6 Assumptions and Dependencies

- It is assumed the calling AI application will supply plain text (not raw model logits or attention weights) as input to the module.
- It is assumed public knowledge APIs (Wikipedia/Wikidata) remain available and reasonably stable in schema for the duration of the project.
- The project depends on availability of an LLM (self-hosted open-source model or a third-party API such as an OpenAI/Anthropic-compatible endpoint) for claim extraction and, optionally, for generating candidate corrections.
- If a licensed live web-search API is unavailable or budget-constrained, the module will fall back to the offline curated corpus and knowledge graph only, with reduced coverage — this is an accepted project risk.

3. External Interface Requirements

3.1 User Interfaces

- Reviewer Dashboard: a web UI showing a list of verification jobs, each expandable to show the original text with claims highlighted in green (Supported), yellow (Not-Checkable), or red (Unsupported/Contradicted), with an evidence panel on hover/click.
- Standard controls: search/filter by date, confidence threshold, and status; an "Approve"/"Override verdict" action for reviewers; a settings screen for administrators to manage trusted sources.
- Error and empty states must be explicit (e.g., "No claims detected in this text" rather than a blank screen).
- Detailed wireframes/mockups are to be documented separately in the UI design specification during the Design phase.

3.2 Software Interfaces

Interface	Purpose
Wikipedia REST API	Retrieve summary/full-text article content used as evidence for general factual claims.
Wikidata Query Service (SPARQL)	Structured knowledge-graph lookups for entities, dates, numeric facts, and relationships.
Curated Document Corpus (vector DB)	Domain-specific or project-supplied trusted documents indexed for semantic similarity search.
LLM Inference API	Used internally for claim extraction (splitting generated text into atomic checkable statements) and for drafting suggested corrections.
PostgreSQL Database	Persists verification history, user/role data, and source configuration.

3.3 Communications Interfaces

- All external and client-facing communication uses HTTPS with TLS 1.2+.
- The primary integration interface is a RESTful JSON API (Content-Type: application/json); an optional WebSocket channel streams claim-by-claim verification results for long documents to support responsive UIs.
- Authentication uses per-integrator API keys (bearer tokens); the dashboard uses session-based authentication for human users.
- Standard HTTP status codes and a consistent JSON error envelope (`{ "error": { "code":, "message": } }`) are used for all failure responses.

4. Analysis Models

Primary use cases to be elaborated with UML use-case and sequence diagrams during the Design phase:

- UC-1: Integrating Developer submits generated text via API and receives an annotated verification report.
- UC-2: Content Reviewer reviews a flagged document in the dashboard and overrides a verdict.
- UC-3: Administrator adds/removes a trusted knowledge source and adjusts the confidence threshold.
- UC-4: System automatically caches a previously verified claim and reuses the cached verdict for an identical/near-duplicate claim to reduce external API calls.

A top-level data-flow diagram (Text Input → Claim Extraction → Parallel Source Verification → Score Aggregation → Annotated Output) and an entity-relationship diagram (User, ApiKey, VerificationJob, Claim, Evidence,

TrustedSource) will be included as diagrams in the Design document (Jackfruit Phase corresponding to Architecture/Design specification).

5. System Features

5.1 Claim Extraction

Description and Priority

Splits a block of AI-generated text into discrete, independently verifiable factual claims (as opposed to opinions, greetings, or non-factual filler). Priority: High.

Stimulus/Response Sequences

- Integrating developer sends raw generated text to the API → system returns a structured list of extracted claim strings with character offsets into the original text.

Functional Requirements

REQ-1: The system shall accept a text input of up to 10,000 characters per verification request.

REQ-2: The system shall segment the input text into a list of discrete factual claims, excluding subjective statements, greetings, and questions.

REQ-3: The system shall preserve, for each extracted claim, the exact character offset range in the original text so the calling application can highlight it in place.

REQ-4: The system shall return an empty claim list (not an error) when no verifiable factual claims are found in the input.

5.2 Fact Verification Engine

Description and Priority

Cross-references each extracted claim against one or more configured trusted knowledge sources and determines whether the claim is supported, contradicted, or cannot be checked. Priority: High.

Stimulus/Response Sequences

- System receives an extracted claim → queries the knowledge graph, curated corpus, and/or web source in parallel → aggregates evidence → returns a verdict with supporting/contradicting evidence snippets.

Functional Requirements

REQ-5: The system shall query at least one structured knowledge source (Wikidata) and one unstructured text source (Wikipedia or curated corpus) for every extracted claim.

REQ-6: The system shall classify each claim as one of: Supported, Contradicted, Unsupported (no matching evidence found), or Not-Checkable (source timeout or non-factual content).

REQ-7: The system shall attach at least one evidence snippet with its source URL/identifier to every claim classified as Supported or Contradicted.

REQ-8: The system shall time out an individual source query after a configurable interval (default 5 seconds) and classify the claim as Not-Checkable for that source without blocking verification of other claims.

5.3 Confidence Scoring and Flagging

Description and Priority

Converts the raw verdicts and evidence quality from the Fact Verification Engine into a single numeric confidence score per claim and an overall document-level hallucination score. Priority: High.

Stimulus/Response Sequences

- System receives evidence and verdicts for a claim → computes a confidence score between 0 and 1 → flags the claim if the score falls below the configured threshold.

Functional Requirements

REQ-9: The system shall compute a confidence score (0.0–1.0) for every processed claim, based on source agreement, evidence recency, and source trust weighting.

REQ-10: The system shall flag any claim with a confidence score below the administrator-configured threshold (default 0.5) as requiring review.

REQ-11: The system shall compute and return an aggregate document-level hallucination score summarizing the proportion and severity of flagged claims.

5.4 Correction and Citation Suggestion

Description and Priority

For claims classified as Contradicted, proposes a corrected version of the statement drawn from the verified evidence. Priority: Medium.

Stimulus/Response Sequences

- System identifies a Contradicted claim with strong supporting evidence for an alternative fact → generates a suggested correction string → returns it alongside the original claim and evidence.

Functional Requirements

REQ-12: The system shall generate a suggested correction for a Contradicted claim only when evidence confidence for the alternative fact exceeds 0.7.

REQ-13: The system shall never silently replace the calling application's original text; suggested corrections shall always be returned as a separate field for the caller or reviewer to accept or reject.

5.5 Integration API / SDK

Description and Priority

Exposes all module capabilities through a documented REST API and a thin client SDK so third-party chatbots and content pipelines can integrate with minimal effort. Priority: High.

Stimulus/Response Sequences

- Developer registers for an API key → calls POST /verify with generated text → receives a JSON verification report synchronously (or via WebSocket stream for long documents).

Functional Requirements

REQ-14: The system shall authenticate every API request using a per-integrator API key and reject unauthenticated requests with HTTP 401.

REQ-15: The system shall provide OpenAPI (Swagger) documentation describing all endpoints, request/response schemas, and error codes.

REQ-16: The system shall enforce a configurable per-key rate limit and return HTTP 429 with a Retry-After header when exceeded.

5.6 Reviewer Dashboard and Analytics

Description and Priority

Provides a human-facing web interface for reviewing flagged content, overriding verdicts, and viewing hallucination-rate trends. Priority: Medium.

Stimulus/Response Sequences

- Reviewer logs in → selects a verification job → views highlighted claims and evidence → approves or overrides a verdict, which is persisted for future audit.

Functional Requirements

REQ-17: The system shall display all claims of a verification job with color-coded status (Supported/Unsupported/Contradicted/Not-Checkable).

REQ-18: The system shall allow an authenticated reviewer to override a claim's verdict, storing the original automated verdict, the override, the reviewer identity, and a timestamp.

REQ-19: The system shall present an analytics view showing hallucination rate over time, filterable by date range and by integrating application/API key.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- For an input of up to 500 characters (roughly 2–3 claims), the system shall return a verification report within 5 seconds under normal network conditions (95th percentile).
- The system shall support at least 20 concurrent verification requests without a queuing delay greater than 10 seconds.
- Cached (previously seen, near-duplicate) claims shall be returned within 500 milliseconds.

6.2 Safety Requirements

- The module shall never automatically publish or transmit AI-generated content on the calling application's behalf; it only annotates and returns results for the caller/reviewer to act on.
- Suggested corrections generated by the module shall be clearly labeled as AI-generated suggestions, not verified fact, until confirmed by a human reviewer or by an evidence-confidence threshold defined in REQ-12.

6.3 Security Requirements

- All API traffic shall be encrypted in transit via TLS 1.2 or higher.
- API keys shall be stored hashed (not in plaintext) in the datastore.
- Role-based access control shall distinguish Administrator, Reviewer, and Integrating Developer privileges within the dashboard and API.
- The system shall log all verdict overrides and configuration changes for audit purposes, retained for a minimum of 90 days.
- Static code analysis (e.g., SonarQube) and dependency vulnerability scanning shall run in the CI/CD pipeline on every pull request, per course requirements.

6.4 Software Quality Attributes

- Maintainability: the Claim Extraction, Fact Verification, Scoring, and API layers shall be independently deployable/testable modules with documented interfaces, so a knowledge source can be added or replaced without modifying other layers.
- Reliability: if any single trusted knowledge source is unreachable, the system shall continue verification using the remaining available sources rather than failing the entire request.
- Usability: the dashboard shall require no training beyond a single onboarding walkthrough for a reviewer to correctly interpret claim status colors and evidence panels.
- Extensibility: adding a new knowledge source shall require implementing a single well-defined source-adapter interface, not changes to the core verification pipeline.
- Testability: every functional requirement in Section 5 shall be covered by at least one automated unit or integration test, per the project's Test Plan (Jackfruit Phase-3).

6.5 Business Rules

- Only users with the Administrator role may add, remove, or reprioritize trusted knowledge sources.
- A verdict override by a Reviewer takes precedence over the automated verdict in all downstream analytics and reports, but the original automated verdict is always retained for audit.
- Free-tier/education API quotas for external knowledge sources shall not be exceeded; the system shall degrade gracefully (returning Not-Checkable) rather than fail when a quota is exhausted.

7. Other Requirements

- Internationalization: Version 1.0 supports English-language input only; the architecture should not preclude adding other languages in a future release.
- Legal/Compliance: the module must clearly attribute all evidence to its original source and respect the usage terms of any third-party API it queries (e.g., Wikipedia/Wikidata attribution and rate-limit policies).
- Reuse objective: the project should reuse open-source components (e.g., an existing sentence-segmentation library, an existing vector-database library) wherever feasible rather than re-implementing them, consistent with course guidance on build-vs-reuse classification (Jackfruit Phase-2).

Appendix A: Glossary

Term	Definition
Hallucination	A statement generated by an AI model that is fluent and plausible but factually incorrect or unsupported by any real source.
Claim	A single, atomic, independently verifiable factual statement extracted from a larger block of text.
Trusted Source	A knowledge graph, document corpus, or web source that the system administrator has configured and weighted for evidence lookup.
Confidence Score	A numeric value (0.0–1.0) representing the system's estimated likelihood that a claim is factually accurate, based on evidence agreement.
Verdict	The classification assigned to a claim: Supported, Unsupported, Contradicted, or Not-Checkable.
Knowledge Graph	A structured database of entities and their relationships (e.g., Wikidata) used for precise fact lookups.

Appendix B: Field Layouts

Sample request/response field layout for the primary verification endpoint (POST /verify):

Field	Length	Data Type	Description	Is Mandatory
text	10,000 (max)	String	The AI-generated text to be verified	Y
source_app_id	64	String	Identifier of the calling application/API key	Y
context	2,000 (max)	String	Optional surrounding context to aid claim extraction	N
confidence_threshold	-	Decimal (0–1)	Overrides the default flagging threshold for this request	N
claim_id	36 (UUID)	String	Unique identifier for an extracted claim	Y
verdict	20	Enum	One of Supported / Unsupported / Contradicted / Not-Checkable	Y
confidence_score	-	Decimal (0–1)	Computed confidence for the claim	Y
evidence_source_url	500	String	URL/identifier of the supporting or contradicting source	N

Report Requirements

Fields to be included in the Reviewer Dashboard's Verification Analytics Report (supports REQ-19):

Verification Analytics Report
Verification Job ID
Source Application / API Key ID
Date and Time of Verification
Total Claims Checked
Supported Claim Count
Unsupported Claim Count
Contradicted Claim Count
Not-Checkable Claim Count
Document-Level Hallucination Score
Reviewer Override Count
Reviewing User (if overridden)

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description of Requirement	Architecture Ref.	Design Ref.	Code File Ref.	Test Case ID
1	REQ-1	Accept text input up to 10,000 chars	TBD	TBD	TBD	UT-01
2	REQ-2	Segment text into discrete factual claims	TBD	TBD	TBD	UT-02
3	REQ-3	Preserve character offsets for each claim	TBD	TBD	TBD	UT-03
4	REQ-4	Return empty claim list when none found	TBD	TBD	TBD	UT-04
5	REQ-5	Query knowledge graph + text source per claim	TBD	TBD	TBD	IT-01
6	REQ-6	Classify claim verdict (4-way)	TBD	TBD	TBD	IT-02
7	REQ-7	Attach evidence snippet to Supported/Contradicted	TBD	TBD	TBD	IT-03
8	REQ-8	Time out a source query and	TBD	TBD	TBD	IT-04

Sl. No	Requirement ID	Brief Description of Requirement	Architecture Ref.	Design Ref.	Code File Ref.	Test Case ID
		continue verification				
9	REQ-9	Compute per-claim confidence score	TBD	TBD	TBD	UT-05
10	REQ-10	Flag claim below confidence threshold	TBD	TBD	TBD	UT-06
11	REQ-11	Compute aggregate document-level hallucination score	TBD	TBD	TBD	UT-07
12	REQ-12	Generate suggested correction above evidence threshold	TBD	TBD	TBD	IT-05
13	REQ-13	Never silently replace caller's original text	TBD	TBD	TBD	UT-08
14	REQ-14	Authenticate API requests via API key	TBD	TBD	TBD	ST-02
15	REQ-15	Provide OpenAPI (Swagger) documentation	TBD	TBD	TBD	ST-01
16	REQ-16	Enforce per-key rate limit (HTTP 429)	TBD	TBD	TBD	ST-03
17	REQ-17	Display claims with colour-coded status	TBD	TBD	TBD	ST-04
18	REQ-18	Reviewer verdict override with audit trail	TBD	TBD	TBD	ST-05
19	REQ-19	Analytics view of hallucination rate over time	TBD	TBD	TBD	ST-06

Note: Architecture/Design/Code references are to be completed once the Architecture & Design Specification and Test Plan (Jackfruit Phase-3) are finalized. Test Case IDs above are provisional and will be confirmed against the actual Test Plan document.